

PlanGo：面向本地生活的可执行规划 Agent

北京邮电大学邓鹏超

一句话概括： PlanGo 将用户的自然语言生活需求转化为可执行的本地行程方案，覆盖 意图理解、上下文治理、约束构建、POI 召回、候选打分、路线组合、可用性检查、失败回退、方案生成与解释。系统目标不是返回 POI 列表，而是生成包含时间线、路线距离、预算估计、风险提示和可替换节点的完整执行计划。

Planning Core Workflow



Figure 1: Planning Core Workflow

1 项目定位

目标场景

- 面向周末、下午、晚间等闲时本地生活规划，当前以北京本地 POI 数据为主要落地对象。
- 用户只需输入模糊自然语言需求，如“今天下午和家人出去玩，别太远，顺便吃饭”。
- 系统自动抽取时间、人群、起点、距离、预算、活动类型、餐饮偏好和避雷约束。
- 输出活动、餐饮、路线、预算、时间线和风险提示，帮助用户完成“去哪、怎么走、是否可行”的决策闭环。

2 Planning Graph 架构

显式编排原则

- 系统采用 LangGraph 构建 Planning Graph，以状态机方式约束 Agent 执行路径。
- 每个节点只负责单一职责，并统一读写 PlanningState，避免跨节点隐式共享和不可审计副作用。
- LLM 主要承担语义理解、会话摘要、召回意图辅助生成和最终文案表达。
- POI 查询、候选评分、候选池平衡、路线组合、可用性过滤和最终排序主要由确定性代码完成。
- Graph 内置分支路由，支持 simple_qa、单类推荐、完整行程规划和方案修改四类请求。

主流程节点

1. **Context Loader**: 加载本轮请求、用户城市、定位、手动起点和 POI 标签目录。
2. **Session Loader**: 读取最近对话、上一轮方案、结构化摘要和当前修改焦点。
3. **Memory Reader**: 注入长期偏好、正负向标签和相似画像摘要。
4. **Intent Resolver**: 识别请求类型、目标类别、场景、人群和初步 slot。
5. **Constraint Builder**: 合并硬约束、软偏好、默认策略和历史上下文。
6. **Recall Compiler**: 将逻辑召回需求编译为受控查询计划，避免 LLM 直接生成 SQL。
7. **Collector & Scorer**: 召回 POI，并按距离、标签、关键词、质量、预算和 Memory fit 打分。
8. **Route Planner**: 组合活动、餐饮、购物、娱乐等 slot，生成候选路线和时间线。
9. **Checker & Ranker**: 检查营业/预约/排队风险，排序输出 Top 方案。
10. **Response Generator**: 生成前端卡片 payload 与自然语言解释。

3 多轮上下文治理

三层上下文模型

- **当前请求上下文**: 包括本轮输入、城市、定位、起点、同步抽取的约束和偏好，优先级最高。
- **会话上下文**: 包括 `recent_turns`、`session_summary`、上一轮方案快照、活跃约束、负向约束和 open questions。
- **长期记忆上下文**: 包括用户长期偏好、避雷标签、正负向偏好簇和相似用户画像摘要。
- 该结构支持多轮指代和增量修改，例如“第二个方案不错，但餐厅换成清淡一点的”。

优先级与压缩策略

- 约束优先级为：当前用户输入 > 当前会话约束 > 上一轮方案状态 > 长期 Memory > 默认值。
- 当前请求中的硬约束，如距离、预算、时间和避雷关键词，不会被长期 Memory 覆盖。
- 长对话会压缩为结构化 `session_summary`，保存 active constraints、negative constraints、current focus、last plan ids 和 open questions。
- Prompt Pack 会进行预算裁剪，只保留摘要、少量 recent turns、少量 memory tags 和上一轮方案 ID。
- 系统不会把完整 POI 原始数据、底层查询、debug 信息或大段经纬度数据直接塞给 LLM。

4 分层 Memory 机制

Memory 定位

- Memory 不是聊天记录缓存，而是长期偏好、行为反馈和个性化排序信号的沉淀层。
- 当前实现通过异步 Memory 事件队列处理用户输入、方案反馈、选择、拒绝、替换和导出等行为。
- 读侧将 Memory 转换为 `positive_tags`、`negative_tags`、preference clusters 和 ranker profile。
- Memory 以软信号参与 POI Scorer 与 Ranker，不直接改变当前请求的硬约束。

Memory 内容与更新

- **用户画像**: 常用城市、默认起点、预算习惯、同行人群、饮食偏好和常见活动类型。
- **结构化偏好**: 喜欢室内、亲子友好、偏好清淡、不喜欢排队、避雷太远、避免太贵或太吵。
- **行为反馈**: 点击、选择、收藏、导出、拒绝方案、替换 POI，以及“太远/太贵/排队”等负向反馈。
- 每条偏好可带 confidence、source、evidence、scene、poi type 等元信息，用于后续置信度更新。
- 当前以文件/runtime store 提供可审计实现；Redis、MySQL 和向量库作为部署增强与语义召回扩展入口。

5 Tool Harness 与工具治理

受控工具调用

- 系统不允许 LLM 直接执行外部操作，而是通过受控节点和 Tool Harness 进行间接调用。
- Tool Harness 统一封装 timeout、retry、fallback、trace、结果标准化和工具缓存摘要。
- Tool Policy 对工具调用进行风险分级：查询类可自动执行，轻状态变更需确认，交易类需幂等键，支付类当前版本禁止自动执行。
- Trace 中只记录脱敏后的工具摘要、参数键、目标 ID、耗时、来源和 fallback 状态，便于审计和调试。

工具与降级机制

- **POI 查询**: 主链路通过受控 Recall Plan 与 Repository 执行, 查询字段、表、limit 和过滤条件由编译器约束。
- **路线工具**: 高德路线服务可作为增强; 不可用、超时或未配置时回退到 Haversine 距离估算。
- **可用性检查**: 基于 POI 字段和 mock 规则检查营业状态、预约可用性、票/桌可用性和排队风险。
- **导出工具**: 支持方案 PDF、预渲染缓存和 ICS 日历导出, 便于将规划结果转化为可保存结果。

6 规划策略与异常处理

召回、打分与候选池平衡

- Recall Compiler 将 slot 需求映射到 restaurant、activity、attraction、shopping、entertainment、fitness、beauty 等逻辑类别。
- Collector 从 MySQL POI 表中召回候选, 并对用户显式指定地点提供 must-keyword 解析和兜底候选。
- Scorer 使用距离、逻辑标签、关键词、质量评分、预算适配和 Memory fit 计算 `final_poi_score`。
- Candidate Pool Balancer 控制类别、价格带和距离带分布, 减少同质化, 提升方案多样性。
- 每个候选保留 score breakdown、推荐理由和 warning, 支撑解释性输出与测试验证。

路线组合与可执行性检查

- Route Planner 基于 slot 笛卡尔组合生成候选方案, 并限制组合扫描规模, 避免候选爆炸。
- 路线阶段会过滤重复地点、语义重复地点、相邻过近地点, 并计算路线距离、交通耗时、访问时长和预算摘要。
- 可行性检查覆盖总时长、最大相邻距离、预算超限、营业关闭、预约不可用和高排队风险。
- Pre-ranker 先保留较优候选进入检查, Post-check 后再由 Final Ranker 输出 Top 方案。
- 最终结果包括 plan slots、timeline、route summary、budget summary、issues 和自然语言解释。

失败分析与回退策略

- Failure Analyzer 将失败归因为 slot 候选不足、标签过严、路线不可行、可用性失败或类别多样性不足。
- Fallback Relaxation 会逐步放宽非核心约束, 包括扩大搜索半径、降低初始评分阈值、增加召回 limit、缩减正向标签限制。
- 若路线不可行, 则适度放宽最大路线时长和总规划时长; 若预算过严, 则放宽软预算边界。
- 回退次数受控, 避免无限重试; 即使外部路线工具失败, 也能通过 fallback 距离估算保证主流程稳定。

工程落地与创新点

- 显式 Planning Graph 将 Agent 拆解为可观测、可测试、可回退的节点链路。
- PlanningState 作为统一状态协议, 使上下文、约束、候选、方案、响应和 debug 信息结构化流转。
- 三层上下文治理支持多轮对话、指代消解和方案修改。
- Memory 以软约束参与个性化排序, 兼顾用户偏好和当前请求的确定性约束。
- Tool Harness 将外部调用纳入可靠性治理和风险控制。
- Event Bus、Trace Recorder、Runtime Store 与 SSE 流式事件支持前端进度展示、运行审计和演示。